

CPT304 Coursework 1 Report

Taskify Research-Led Software Enhancement

Group 13

May 2026

1 App Selection

We chose Taskify because it already behaved like a real web application, but it still had obvious weaknesses once we started testing it properly. The project had an Express backend, authentication pages, a dashboard, automated tests, and a live Vercel deployment, so we could inspect the code, reproduce the problems in the browser, and keep evidence for each change. In the first audit, we identified four specific deficiencies: placeholder-only form fields, missing backend form handlers, dashboard state loss after refresh, and weak task feedback. That made the project suitable for this coursework because the issues were real enough to analyse, but still small enough to fix and verify within the report.

Direct GitHub PR links for verification. Auth form accessibility: <https://github.com/maoyouaa/Taskify/pull/1>; dashboard persistence: <https://github.com/maoyouaa/Taskify/pull/3>; task feedback/actions: <https://github.com/maoyouaa/Taskify/pull/4>; auth form POST routes: <https://github.com/maoyouaa/Taskify/pull/5>; bilingual language toggle evidence: <https://github.com/maoyouaa/Taskify/pull/6>.

2 Authentication Form Fields Relied on Placeholder Text

2.1 Detection

The first problem was detected with browser DevTools inspection of the rendered sign-up form and its markup. Each account field used placeholder text as the only visible label. That meant the field name disappeared as soon as the user started typing, which is awkward even in a simple form. We typed into the username field, observed the placeholder vanish in the browser, and then checked all five account inputs to confirm that the same issue affected `SignUpUsername`, `SignUpEmail`, `SignUpPassword`, `LoginEmail`, and `LoginPassword`. Figure 1 records that before-and-after state for verification.

2.2 Literature

The accessibility sources we used all pointed in the same direction. W3C WAI's forms tutorial recommends a visible label that is tied to the input with matching `for` and `id` attributes [1]. WebAIM makes the practical point that placeholders are useful as hints,

but they are not a substitute for labels because they disappear while the form is being filled in [2]. W3C's explanation of WCAG 2.2 Success Criterion 3.3.2 says the same thing in a more formal way: labels should remain available as instructions, not vanish with the placeholder [3]. That was the basis for the fix.

2.3 Implementation

The original email field depended on the placeholder for its meaning:

Before fix: placeholder-only email field.

```
<input type="email" name="SignUpEmail"
  placeholder="Email" required="true" />
```

We changed the form so the label stays visible and the hint becomes secondary:

After fix: visible label connected to the input.

```
<label class="field-label" for="signup-email">Email</label>
<input id="signup-email" type="email" name="SignUpEmail"
  autocomplete="email" placeholder="Email" required />
```

The same pattern was applied to the other account fields in the sign-up template. We also split the CSS so `.toggle-label` still styles the Sign Up / Log In switch, while `.field-label` styles the ordinary form labels. After that change, the form kept a stable label after typing, and the accessibility tree exposed clearer names such as `textbox "User name"` and `textbox "Email"`.

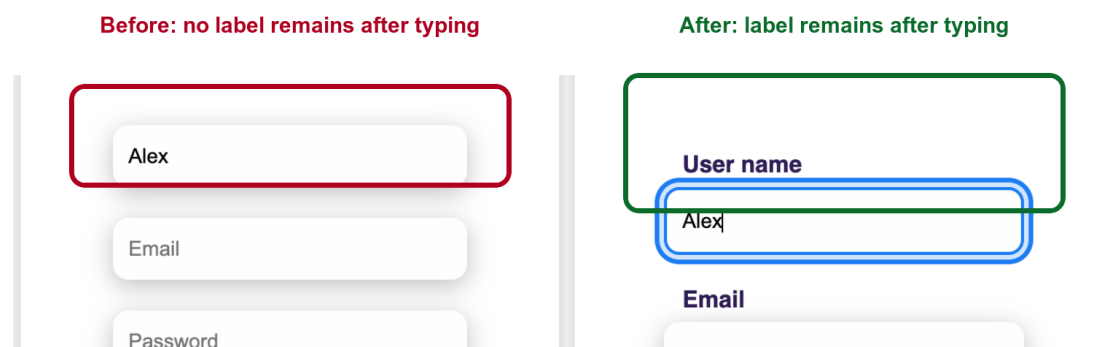


Figure 1: Authentication form label evidence. Before the fix, the username placeholder disappears after typing and no field label remains. After the fix, the visible “User name” label stays above the filled input.

3 Authentication Forms Submitted to Missing Backend Routes

3.1 Detection

The second deficiency appeared when we compared the frontend forms with the server routes. The page submitted `POST /signup` and `POST /login`, but the earlier server setup only defined `GET` routes for the page views. That meant the interface looked complete even though the backend could not process either form. We confirmed this by sending a signup request and receiving `HTTP/1.1 404 Not Found`, as shown in Figure 2.

3.2 Literature

The implementation decision here came from a mix of framework documentation and security guidance. Express explains that routes are matched by both path and HTTP method, so GET /signup cannot handle a POST /signup submission [4]. OWASP's Input Validation Cheat Sheet also states that browser-side checks are not enough on their own, because the server still needs to validate untrusted input [5]. Finally, MDN's explanation of 303 See Other gave us a clean way to return the user to a GET page after a successful POST request [8].

3.3 Implementation

Originally, the sign-up page could render, but it could not accept its own form submission:

Before fix: sign-up page existed, but no POST handler existed.

```
app.get("/signup", (req, res) => {
  res.status(200).render("signup.ejs");
});
```

We added explicit POST handlers with server-side validation and redirect logic:

After fix: server-side validation and Post/Redirect/Get.

```
app.post("/signup", (req, res) => {
  const missingFields = collectMissingFields(req.body, [
    "SignUpUsername", "SignUpEmail", "SignUpPassword",
  ]);

  if (missingFields.length > 0 || !isValidEmail(req.body.SignUpEmail)) {
    return res.status(400).json({
      success: false,
      errors: {
        missingFields,
        email: isValidEmail(req.body.SignUpEmail) ? undefined
          : "Enter a valid email address.",
      },
    });
  }

  return res.redirect(303, "/dashboard");
});
```

The same approach was added for POST /login. After the change, a valid signup request returned 303 See Other with Location: /dashboard, while an invalid email produced 400 Bad Request and an error message. The later hardening work then extended this flow by storing registered-user passwords as hashes and requiring a signed session cookie before /dashboard could be opened [6, 7]. We also exported the app and only called listen() when run directly, so the route tests could run against a temporary server.

Backend route evidence

Before fix:

```
POST /signup -> HTTP/1.1 404 Not Found
```

After fix:

```
POST /signup -> HTTP/1.1 303 See Other  
Location: /dashboard
```

```
POST /login -> HTTP/1.1 303 See Other  
Location: /dashboard
```

```
Invalid email -> HTTP/1.1 400 Bad Request  
{ "success": false, "errors": { "email": "Enter a valid email address." } }
```

Figure 2: Backend route evidence. Before the fix, `POST /signup` returned 404. After the fix, valid submissions redirect to `/dashboard` with 303, while invalid email input returns 400.

4 Dashboard Tasks Were Lost After Refresh

4.1 Detection

This deficiency was detected through a browser refresh test and `localStorage` inspection. We added and edited tasks on `/dashboard`, refreshed the page, and compared the rendered board with the stored task payload. The board returned to the seeded example data, which meant the interface gave the impression that tasks had been saved even though the state only existed in memory. When we then checked the dashboard script, the reason was obvious: `state.tasks` was rebuilt from `seedTasks` on every page load and nothing wrote the latest task array to persistent storage.

4.2 Literature

The research for this fix was quite straightforward. MDN explains that `localStorage` keeps browser data for the same origin across refreshes and later sessions, so it is suitable for lightweight client-side persistence [9]. `web.dev` adds an important warning: stored data should not be trusted blindly, and the application should recover safely when the stored value is missing or malformed [10]. We took those two points as the design rule for this deficiency: save the task array after each change, but validate any stored data before using it.

4.3 Implementation

In the earlier version, the dashboard always started from the same built-in task list:
Before fix: dashboard state was recreated on every load.

```

let state = {
  tasks: seedTasks.map((task) => ({ ...task })),
};

function deleteTask(taskId) {
  state.tasks = state.tasks.filter((task) => task.id !== taskId);
  renderBoard();
  showMessage("Task deleted successfully.");
}

```

We moved persistence into a small module and restored saved tasks when valid browser storage was available:

After fix: state is loaded safely and saved after changes.

```

import { persistTasks, safeLoadTasks } from "../modules/task-store.mjs";

let state = {
  tasks: safeLoadTasks(window.localStorage,
    seedTasks.map((task) => ({ ...task }))).tasks,
};

function persistState() {
  persistTasks(window.localStorage, state.tasks);
}

function deleteTask(taskId) {
  state.tasks = state.tasks.filter((task) => task.id !== taskId);
  persistState();
  renderBoard();
}

```

The storage key is `taskify.dashboard.tasks`. The verification trace gave a concrete result rather than a vague claim: Injected task count: 1, Task cards after reload: 1, and the same JSON payload still present after refresh. The restored task also kept the title Persist after refresh, status doing, priority high, owner Member D, and due date 2026-05-11. That matters because it shows that both the rendered board and the stored data survived the reload. In code, `safeLoadTasks()` wraps `JSON.parse` in a guarded fallback path, while `persistState()` runs after save, delete, and move actions. We also switched the dashboard script to `type="module"`, and dedicated module tests cover the missing-storage and malformed-JSON cases.

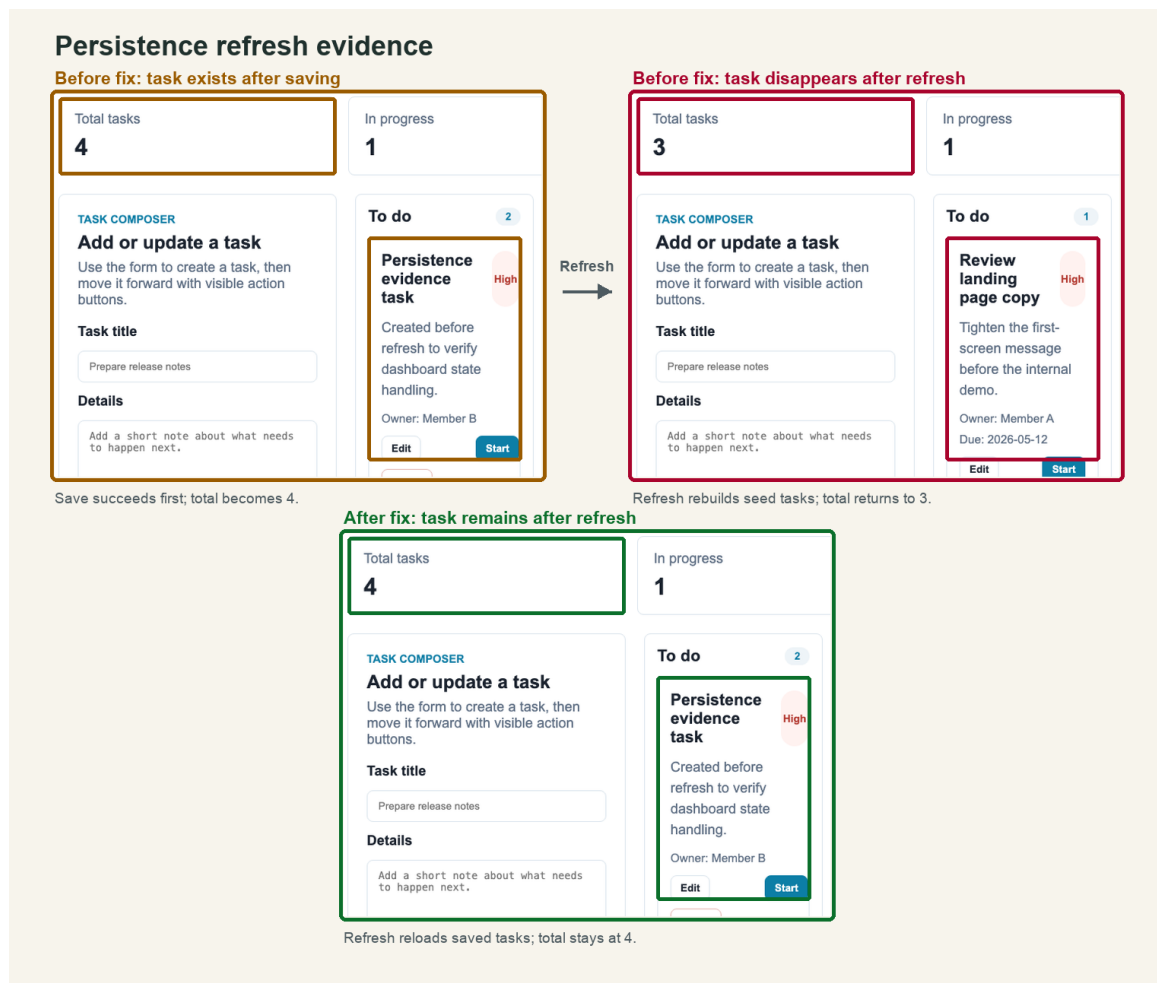


Figure 3: Persistence refresh evidence. Before the fix, refresh rebuilt the board from seed tasks and removed the saved task. After the fix, the same refresh reloads the `localStorage` task list and keeps the task visible.

5 Weak User Feedback and Unclear Task Actions

5.1 Detection

This issue was identified through browser interaction testing, returned-HTML inspection, and source review of the earlier dashboard templates and scripts. In the browser, the old task cards looked presentable but offered very little direct control. The earlier interface had no dedicated live feedback area, and actions such as edit, move, reset, or delete were not exposed as obvious per-card controls. As a result, users had to guess whether something had happened by watching for small layout changes. In a task manager, that is a poor interaction pattern because routine actions should be explicit and easy to confirm.

5.2 Literature

The research here focused on general usability principles rather than one narrow technical bug. Barrington argues that usable systems should communicate system state

clearly and provide actions that users can understand and recover from [11]. Nielsen Norman Group describes similar ideas through the heuristics of visibility of system status and user control and freedom [12]. Taken together, these sources suggest that a task board should not just display information. It should also show what the user can do next and confirm the result immediately.

5.3 Implementation

Before the change, the dashboard was closer to a static task layout than a fully interactive workspace:

Before improvement: static dashboard content with limited user action flow.

```
<div class="dashboard-task task-1">
  <div class="dashboard-task-head">
    Lorem ipsum dolor sit amet.
  </div>
  <div class="dashboard-task-description">
    Lorem ipsum dolor sit amet consectetur adipisicing elit.
  </div>
</div>
```

We changed the task cards so they expose clear actions and the page responds to each important change with visible feedback:

After improvement: explicit task actions and visible feedback.

```
actions.append(
  createActionButton("Edit", () => {
    fillForm(task);
    form.elements.title.focus();
    showMessage("Task loaded for editing.");
  })
);

if (task.status === "todo") {
  actions.append(createActionButton(
    "Start",
    () => moveTask(task.id, "doing", "Task moved to In progress."),
    "primary"
  ));
}
```

Most of this work was done in the client-side dashboard logic, supported by related template and CSS updates. An authenticated verification trace makes the result measurable: after an authenticated signup flow, the signup response is 303, the redirected dashboard response is 200, and the returned HTML contains `id="form-message"`, `id="metric-total"`, `id="task-form"`, `id="board-grid"`, `dashboard-button-primary`, and `dashboard-button-secondary`. The resulting dashboard, shown in Figure 4, renders visible controls such as Edit, Start, Complete, Reset, and Delete. This brought the interface closer to what the literature recommends: actions are explicit, feedback is immediate, and the user is not left guessing whether the board accepted the change. The figure captures a concrete example: clicking Start moves the task to *In progress*, updates the counters, and replaces the default message with a confirmation message.

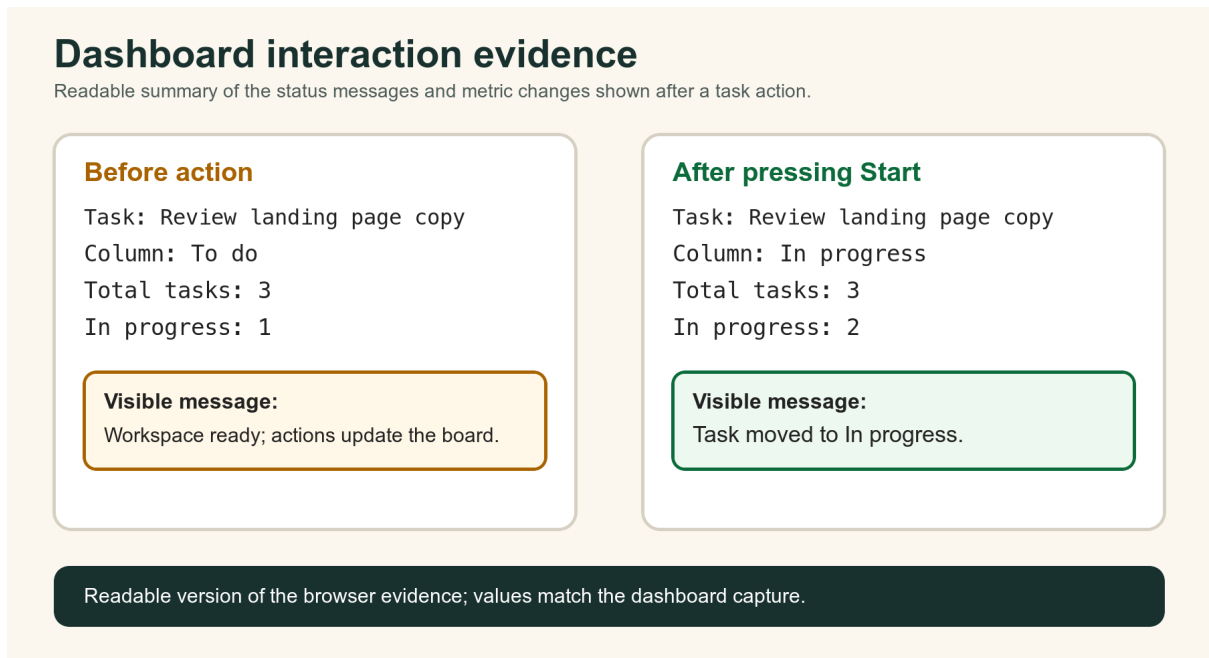


Figure 4: Dashboard interaction evidence. Before the fix, the task remained in the To do column and the interface showed only the default message. After Start, the task moves to In progress, a confirmation message appears, and the metrics update.

6 Baseline Standards Evidence

The baseline standards were documented separately from the four deficiencies, as required by the brief. Figure 5 shows the Vercel production history captured on 2026-05-17, including deployments aged 7d and 17d and a same-day uptime confirmation from the live site. Figure 6 shows repository coverage evidence above the required 80% threshold. Figure 7 records a Lighthouse Accessibility score of 100. Figure 8 shows the Chinese homepage reached through the working language toggle. Figures 9 and 10 confirm the legal baseline through a visible cookie banner and a dedicated privacy-policy page.

Taskify Vercel Deployment History

Public deployment history captured on **2026-05-17** using `npx.cmd --yes vercel ls cpt304-taskify-coursework`.

Evidence of the coursework uptime window appears in the production rows at **7d** and **17d**, with the project still reachable at the current production alias.

```
cmd.exe vercel deployment history excerpt

Fetching deployments in g3519230338-3646s-projects
> Deployments for g3519230338-3646s-projects/cpt304-taskify-coursework [574ms]

Age      Project
Status   Environment  Duration  Deployment
7d       g3519230338-3646s-projects/cpt304-taskify-coursework https://cpt304-taskify-coursework-m3njy8hw4-g3519230338-3646s-projects.vercel.app • Ready Production 10s
7d       g3519230338-3646s-projects/cpt304-taskify-coursework https://cpt304-taskify-coursework-o504awmk4-g3519230338-3646s-projects.vercel.app • Ready Production 5s
17d      g3519230338-3646s-projects/cpt304-taskify-coursework https://cpt304-taskify-coursework-4fubd4jae-g3519230338-3646s-projects.vercel.app • Ready Production 13s
1h       g3519230338-3646s-projects/cpt304-taskify-coursework https://cpt304-taskify-coursework-e6uncts41-g3519230338-3646s-projects.vercel.app • Ready Production 8s

Alias in use
https://cpt304-taskify-coursework.vercel.app
```

Figure 5: Vercel deployment-history evidence captured on 2026-05-17. The production log shows deployment ages of 7d and 17d, and the separate uptime check for the same date returned 200 OK from the production alias.

Taskify Codecov Coverage Evidence

Live badge source: <https://codecov.io/gh/maoyouaa/Taskify/graph/badge.svg>
Coverage summary below mirrors the repository verification stored in `docs/baseline/evidence/coverage-summary.txt`.

Repository badge available on the public Codecov project page
The badge resolves successfully and can also be shown in the README for marker verification.

Statements
92.18%

Branches
84.07%

Tests passed
26 / 26

Verification command
npm run coverage

Figure 6: Test-coverage evidence. This repository evidence image combines the delivered project's live Codecov badge with the current coverage-summary figures, showing coverage above the required 80% threshold.

9

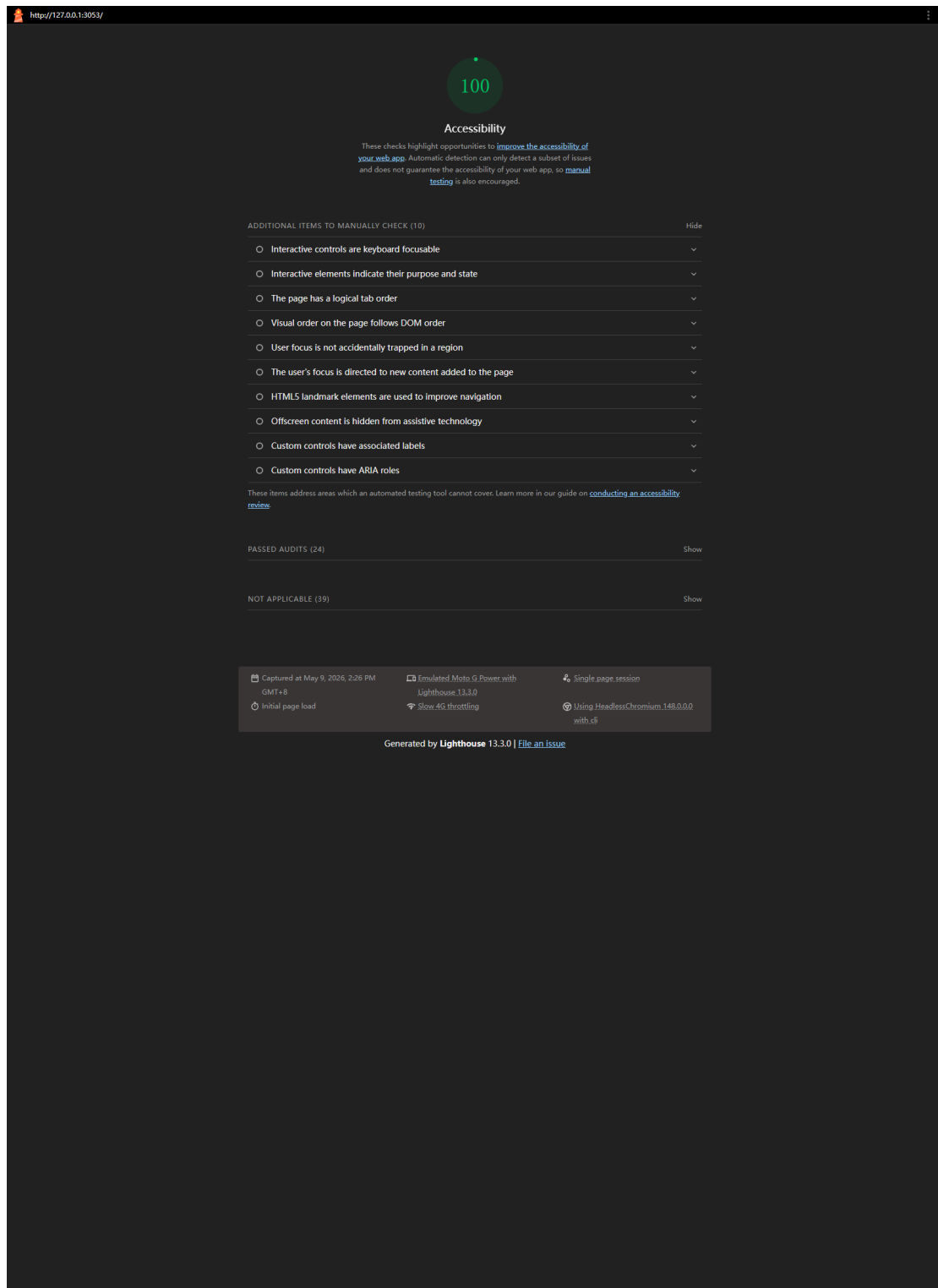


Figure 7: Accessibility evidence. Lighthouse reports an Accessibility score of 100 for the delivered interface, which exceeds the required 90+ baseline.

一个应用替代所有任务工具

输入你的邮箱地址

立即开始



简单顺手，却足够强大的任务管理

把工作、学习和生活安排到一个地方，不再被杂乱流程拖慢节奏。



随时随地跟着你

在任意设备上使用 Taskify，让待办始终触手可及。

下载应用



从模板快速起步

直接套用适合团队协作、学习计划和个人目标的任务布局。

使用模板



打造适合你的流程

通过筛选、标签、优先级和简洁视图来定制任务管理。

查看全部功能



连接常用工具

把 Taskify 与日历和团队常用应用联动起来。

安装集成



建立更稳的习惯

用轻量规划保持专注，让效率提升更自然。

开始测评

值得长期依赖的任务管理器

Taskify 用更清晰、更稳定的流程帮助团队整理项目。



300+
应用下载量



100+
完成任务数



160+
合作院校



100+
专业版用户

加入数百万人，一起用 Taskify 管理工作与生活。



功能

工作原理
团队方案
价格
模板

资源

下载应用
帮助中心
效率方法
邀请好友
集成
渠道合作
开发者 API
系统状态

公司

关于我们
加入我们
博客
媒体

One app to replace them all

[Get Started](#)


Delightfully simple and deceptively powerful task management

Plan work, study, and life in one place without the usual clutter.



With you everywhere

Use Taskify on any device so your task list stays close at hand.



Start from a template

Jump into proven task layouts for team work, study, and personal goals.

[Demo](#)

COOKIE SETTINGS

Choose how this demo remembers your preferences.

We use a small consent cookie to remember your choice and local storage to keep task data on this device.

[Necessary only](#)
[Reject optional](#)
[Accept all](#)


Shape a workflow that fits

Customize tasks with filters, labels, priorities, and simple views.

[See all features](#)

Connect your other tools

Keep Taskify close to your calendar and the apps your team already uses.

[Install integrations](#)


Build better habits

Use lightweight planning to stay productive without losing focus.

[Take the quiz](#)

A task manager you can trust for life

Taskify keeps projects organized with a calmer, clearer workflow for every team.



300+

App downloads



100+

Tasks completed



160+

Colleges



100+

Pro users

Join millions of people who organize work and life with Taskify.



FEATURES

[How It Works](#)
[For Teams](#)
[Pricing](#)
[Templates](#)

RESOURCES

[Download Apps](#)
[Help Center](#)
[Productivity Methods](#)
[Refer a friend](#)
[Integrations](#)
[Channel Partners](#)
[Developer API](#)
[Status](#)

COMPANY

[About Us](#)
[We are hiring!](#)
[Blog](#)
[Press](#)

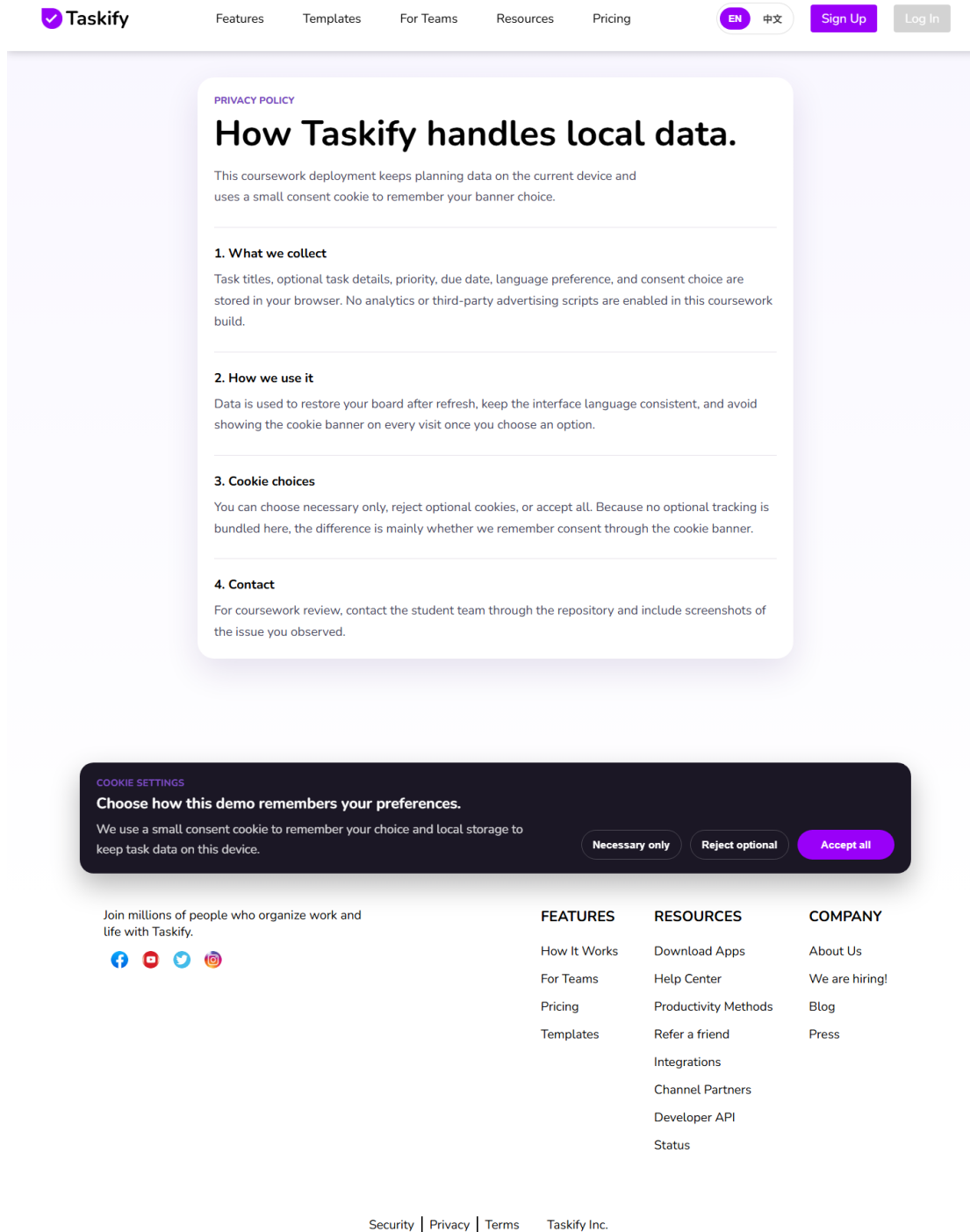


Figure 10: Privacy-page evidence for the legal-compliance baseline. The delivered application includes a dedicated privacy-policy page alongside the cookie banner.

7 Individual Contribution Forms

Group ID	13													
Project Name	Taskify													
GitHub URL	https://github.com/maoyouaa/Taskify													

Teammate Info			
#	Teammate Name	Teammate ID	GitHub Username
1	Cheng Guorui	2253189	maoyouaa
2	Sun Yifan	2255617	peterlololss
3	Xie Zhuoyan	2254529	juiSeIn
4	Zhang Shuo	2252868	MinONclm

Quantitative Peer Assessment				
	Cheng Guorui	Sun Yifan	Xie Zhuoyan	Zhang Shuo
Cheng Guorui	100	100	100	100
Sun Yifan	100	100	100	100
Xie Zhuoyan	100	100	100	100
Zhang Shuo	100	100	100	100
Total	400/400	400/400	400/400	400/400

Individual Technical Evidence					
Deficiency Title	Contribution %				
	Cheng Guorui	Sun Yifan	Xie Zhuoyan	Zhang Shuo	Total
Authentication Form Fields Relied on Placeholder	100	0	0	0	100/100
Authentication Forms Submitted to Missed	0	100	0	0	100/100
Dashboard Tasks Were Lost After Refresh	0	0	100	0	100/100
Weak User Feedback and Unclear Task Assignment	0	0	0	100	100/100

Baseline Standards					
Baseline Standards	Contribution %				
	Cheng Guorui	Sun Yifan	Xie Zhuoyan	Zhang Shuo	Total
1. Live Uptime	35	25	20	20	100/100
2. Test Coverage	10	25	45	20	100/100
3. Accessibility	30	30	15	25	100/100
4. i18n	30	20	20	30	100/100
5. Legal Compliance	20	25	25	30	100/100

Add your signatures here

8 References

- [1] W3C Web Accessibility Initiative, "Labeling Controls," Forms Tutorial. [Online]. Available: <https://www.w3.org/WAI/tutorials/forms/labels/>. Accessed: May 8, 2026.
- [2] WebAIM, "Creating Accessible Forms - Advanced Form Labeling." [Online]. Available: <https://webaim.org/techniques/forms/advanced>. Accessed: May 8, 2026.
- [3] W3C Web Accessibility Initiative, "Understanding Success Criterion 3.3.2: Labels or Instructions." [Online]. Available: <https://www.w3.org/WAI/WCAG22/Understanding/labels-or-instructions.html>. Accessed: May 8, 2026.
- [4] Express, "Routing." [Online]. Available: <https://expressjs.com/en/guide/routing.html>. Accessed: May 8, 2026.
- [5] OWASP Cheat Sheet Series, "Input Validation Cheat Sheet." [Online]. Available: https://cheatsheetseries.owasp.org/cheatsheets/Input_Validation_Cheat_Sheet.html. Accessed: May 8, 2026.
- [6] OWASP Cheat Sheet Series, "Authentication Cheat Sheet." [Online]. Available: https://cheatsheetseries.owasp.org/cheatsheets/Authentication_Cheat_Sheet.html. Accessed: May 14, 2026.
- [7] OWASP Cheat Sheet Series, "Session Management Cheat Sheet." [Online]. Available: https://cheatsheetseries.owasp.org/cheatsheets/Session_Management_Cheat_Sheet.html. Accessed: May 14, 2026.
- [8] MDN Web Docs, "303 See Other." [Online]. Available: <https://developer.mozilla.org/en-US/docs/Web/HTTP/Reference/Status/303>. Accessed: May 8, 2026.
- [9] MDN Web Docs, "Window: localStorage property." [Online]. Available: <https://developer.mozilla.org/en-US/docs/Web/API/Window/localStorage>. Accessed: May 9, 2026.
- [10] web.dev, "Storage for the web." [Online]. Available: <https://web.dev/storage-for-the-web/>. Accessed: May 9, 2026.
- [11] S. Barrington, "Usability in the Lab: Techniques for Creating Usable Products," *Journal of Laboratory Automation*, vol. 12, no. 1, 2007. [Online]. Available: <https://doi.org/10.1016/j.jala.2006.08.004>. Accessed: May 8, 2026.
- [12] Nielsen Norman Group, "10 Usability Heuristics for User Interface Design." [Online]. Available: <https://www.nngroup.com/articles/ten-usability-heuristics/>. Accessed: May 17, 2026.